



WELCOME

vLLM Inference Meetup

Bengaluru

19 September 2026

HOSTED BY

AMD, NextGen, Red Hat

InferenceOps

vLLM Inference Meetup / Bengaluru 2026

1

Agenda

Time	Session	Speaker
1:15-1:30 PM	Welcome Note	Prasad Mukhedkar
1:30-1:45 PM	Keynote 1: Sovereign AI Inference	Sudhir Dharanendraiah Senior Manager, Pyto
1:45-2:00 PM	Keynote 2: Tokenomics and Private AI Inference	Ramakrishna Yekulla
2:00-2:30 PM	Scaling Inference at NxtGen Using the vLLM ecosystem	Abhishek Kumar Singh
2:30-3:00 PM	llm-d for Sovereign & Agentic Workloads	Pravein Govindan Kannan
3:00-3:30 PM	Distributed Inference on ROCm WideEP on vLLM & llm-d	Chaitanya Sri Krishna Lolla & Sirra Ajith
3:30-4:00 PM	Coffee & snacks	
4:00-4:30 PM	Inside vLLM Semantic Router Intelligent routing for LLM inference	Aayush Saini
4:30-5:00 PM	Alligator.sh Powered by vLLM inference	Ompragash Prasad
5:00-6:00PM	Hands-on Workshop with Real GPU	AMD Team

2

From Serving Models to Scaling Models

SERVE

vLLM

Think Docker / Podman

Make one inference server extremely efficient.

- Continuous batching
- KV-cache management
- High GPU utilization

WHAT CHANGES AT SCALE?

100s/1000s GPUs
not just one server

Many replicas
where should each request go?

KV cache
can work be reused?

Prefill + decode
should they scale independently?

SCALE

llm-d

Think Kubernetes

Turn many inference servers into one system.

- Intelligent request routing
- Distributed / disaggregated serving
- Multi-node orchestration

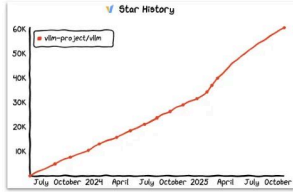
The shift is not just more GPUs – it is from optimizing an engine to orchestrating a system.

vLLM makes each inference engine efficient. llm-d coordinates them at scale.

3

vLLM

The High-Throughput and Memory-Efficient inference and serving engine for LLMs



Most Popular LLM Serving Engine

- PyTorch Foundation Hosted Project
- 89K+ GitHub stars, 800+ PRs/month
- 500K++ GPUs deployed 24/7
- 3K+ contributors, 10K+ members in slack.vllm.ai

>2000 Contributors from >50 major companies



<https://github.com/vllm-project/vllm>

```
$ uv pip install vllm --torch-backend=auto
$ vllm serve deepseek-ai/DeepSeek-V3.1 -tp 8
```

Broad Model Support (>100 arches)



Flexible Device Parallelism

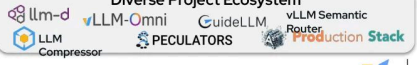
Tensor, Pipeline, Expert, Data, Context Parallel

Disagg Prefill/Decode, Disagg Encoder

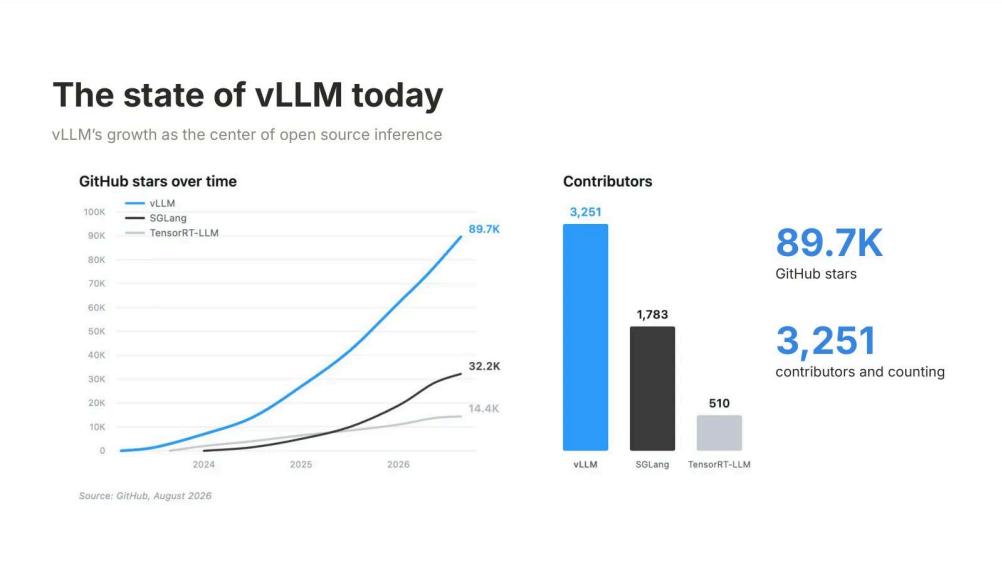
Wide Hardware Support



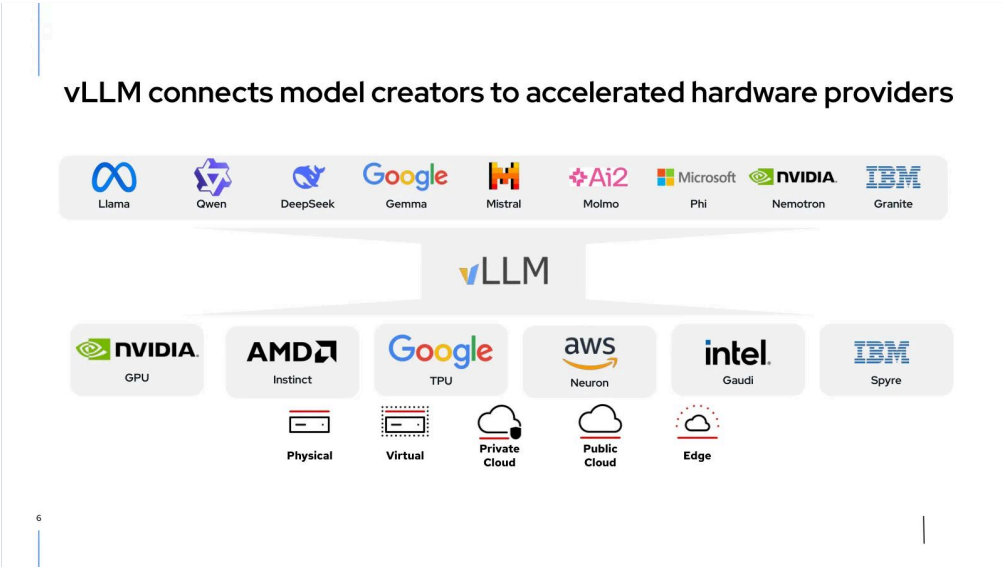
Diverse Project Ecosystem



4



5



6

Bookmark1: <https://recipes.vllm.ai/>

vLLM Recipes

Search models or providers... 36K

Browse Docs GitHub

LATEST RECIPES

- Intern-S2-397B** (Intern) - 37B/17B, BF16, 262K ctx, Multimodal. Flagship scientific multimodal MoE (397B total / 17B active) on the Qwen3.5 hybrid full-attention...
- Owen3.8-Flash-Next** (Owen) - 17B/6.6, NVFP4, 262K ctx, Multimodal. Qwen3.8 architecture preview with a 128B-parameter main model, supplemented by an additional 51B N...
- DeepSeek-V4.1-Flash** (DeepSeek) - 92B/5-14B, FP8, 1B ctx, Text. DeepSeek V4.1 Flash vision-language MoE (52B backbone, 8B active per prompt token, 16B per outp...
- Ling-3.0-flash** (Ling) - 124B/5.5B, BF16, 262K ctx, Text. Ling 3.0-Flash MoE model with BF16, FP4, and INT4 checkpoints, native MTP, and an external DSPar...
- Ling-3.0-flash-VL** (Ling) - 124.35B/5.51B, BF16, 113K ctx, Text. Ling 3.0-flash vision-language MoE model with BF16, FP4, and INT4 checkpoints
- MiniCPM5-2B** (MiniCPM) - 2B, BF16, 113K ctx, Text. MiniCPM5-2B — dense 2B LLM with hybrid Think/No-Think reasoning, native 128K context, and native tool...
- DeepSeek-V4-Flash-Vision-Exp** (DeepSeek) - 285B/13B, FP8, 1B ctx, Text. DeepSeek's first experimental multimodal V4 model — the V4-Flash MoE backbone plus a 32-layer vision...
- K2-Horizon-0.9B** (K2) - 1.88B, BF16, 113K ctx, Text. Compact dense model for mathematics, code, instruction following, and STEM tasks

BROWSE BY PROVIDER

- Arcee AI (1 recipe)
- Ernie (Baidu) (3 recipes)
- Boson AI (1 recipe)
- Seed (ByteDance) (1 recipe)
- DeepSeek (11 recipes)
- Dots (1 recipe)

7

Bookmark2: <https://llm-d.ai/docs/well-lit-paths>

Tested, benchmarked recipes for running LLM inference workloads in production

Intelligent Inference Scheduling KV cache locality estimation routing, load-aware routing	"Precise" prefix aware routing prefix match against KVEvents API routing
Prefill-Decode Disaggregation Independent pools per inference phase	KV Cache Management KV-offloading, P2P KV-sharing
Batch Serving Maximize throughput for offline inference	Multi-Node "Wide Expert Parallel" Serving DP attention + EP MoE across nodes
Flow Control Multi-tenancy, fairness-scheduling	Workload Variant Autoscaling Scale variants to workload demand
Reinforcement learning Scale rollout generation without slowing training	Multimodal Serving Preserve reusable reasoning context across image and audio generation
Agentic Serving – workload-centric Start from the workload, compose the capabilities it needs per accelerator	

8

KEYNOTE 01

Sovereign AI Inference

Sudhir Dharanendraiah

Red Hat

vLLM Inference Meetup / Bengaluru 202603

9

A simple question

Where does your AI actually live?

If inference leaves your control, can you really call the AI sovereign?

The model is only one layer of the system.

SOVEREIGN AI INFERENCE · vLLM INFERENCE MEETUP · BENGALURU · 2026

10

Sovereignty is a systems property

It is not a checkbox on the model card.

CONTROL

CHOICE

TRUST

ECONOMICS

CONTINUITY

Where inference runs

Which model + accelerator

What code actually executes

Who controls token cost

Can you operate without a single vendor

Sovereignty = freedom to choose, control, verify and operate.

SOVEREIGN AI INFERENCE · vLLM INFERENCE MEETUP · BENGALURU · 2026

11

Inference changes the sovereignty equation

Training creates capability. Inference turns it into an operating system for intelligence.

TRAINING

INFERENCEDATA

APPLICATION

Weights Data Compute

Latency Throughput Routing Caching Observability

Agents Products Decisions Workflows

The strategic question becomes: who controls the serving layer?

SOVEREIGN AI INFERENCE · vLLM INFERENCE MEETUP · BENGALURU · 2026

12

The open inference stack

05

A sovereign stack is not one product. It is a set of interoperable layers.

APPLICATIONS & AGENTS	Where intelligence creates value
DISTRIBUTED INFERENCE	Routing · scheduling · autoscaling · KV-cache awareness
INFERENCE ENGINE	vLLM · kernels · compilation · quantization
MODEL & ARTIFACTS	Open models · weights · provenance · policy
ACCELERATORS	NVIDIA · AMD · Intel · other platforms
FOUNDATION	Linux · Kubernetes · observability · security

SOVEREIGN AI INFERENCE · vLLM INFERENCE MEETUP · BENGALURU · 2026

vLLM docs · llm-d docs

13

vLLM makes inference fast. The next challenge is making it sovereign at scale.

06

Think engine + distributed operating layer.

vLLM

- High-throughput serving
- PagedAttention + KV-cache efficiency
- Continuous batching
- Compilation + optimized kernels
- Distributed execution

llm-d

- Kubernetes-native distributed inference
- Intelligent routing
- Autoscaling + resource efficiency
- Observability
- Production-scale orchestration

Open inference infrastructure becomes a strategic capability.

SOVEREIGN AI INFERENCE · vLLM INFERENCE MEETUP · BENGALURU · 2026

vLLM · llm-d official docs

14

Sovereignty requires hardware freedom

07

A sovereign architecture should not turn into a hardware lock-in.

MODEL

INFERENCEDISTRIBUTED INFERENCE

ACCELERATOR

INFRASTRUCTURE

Open models

Portable serving

Multiple accelerators

Hybrid cloud

Choice is not an implementation detail. It is part of sovereignty.

The goal is not to eliminate every dependency — it is to avoid dependencies you cannot change.

SOVEREIGN AI INFERENCE · vLLM INFERENCE MEETUP · BENGALURU · 2026

15

Sovereign inference must be verifiable

08

Fast inference is not enough.

PROVENANCE

What model, code and artifacts entered production?

REPRODUCIBILITY

Can we recreate the serving environment?

SECURITY

Can we inspect, patch and validate the stack?

OBSERVABILITY

Can we see performance, failures and resource use?

Trust comes from the ability to inspect and reproduce — not simply from where the software came from.

SOVEREIGN AI INFERENCE · vLLM INFERENCE MEETUP · BENGALURU · 2026

16

The next frontier: distributed intelligence

09

Inference is becoming a distributed system.



The opportunity is to make intelligence portable, composable and locally controllable.

SOVEREIGN AI INFERENCE · vLLM INFERENCE MEETUP · BENGALURU · 2026

17

Sovereignty is not about building everything yourself.

10

It is about having the freedom
to choose, control, verify and operate.

Open source gives us the foundation.
Open inference gives us the leverage.
Engineering gives us the sovereignty.

Thank you.

SOVEREIGN AI INFERENCE · vLLM INFERENCE MEETUP · BENGALURU · 2026

18

KEYNOTE 02

Tokenomics and Private AI Inference

Ramakrishna Yekulla

Red Hat

vLLM Inference Meetup / Bengaluru 2026

19

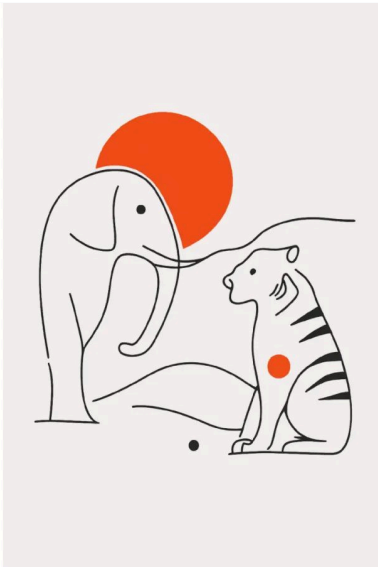


Quick show of Hands.

20

The Token Raj :

How frontier AI vendors colonized your inference stack



21

CHAPTER 1: PRICING

The Subsidy Is Over

Frontier labs kept token prices unsustainably low to grab market share. That window has closed. Capex is at record highs, and the bill is coming — **you're the one paying it.**

\$12/M

Google Gemini

Triples in price each year. Aggressive land-grab pricing now reversing.

\$25/M

Anthropic Claude

Premium pricing maintained. Recent cuts only on top-tier models.

\$30/M

OpenAI GPT-4o

Flagship was subsidized, then flipped upward as margins tightened.

"Cuts when cash is plentiful & share matters. Increases when cash is tight & margins matter." — The latter is now the case for all three vendors simultaneously.

<https://thetangos.com/model-inflation/>

22

CHAPTER 2: TOKENFLATION

Your Tokens Are Shrinking

Bigspin built a **Consumer Price Index for AI coding output** — tracking 5,851 real Opus 4.6 sessions over 10 weeks. Same model. Same tasks. A token in mid-April bought just **\$0.23 on the February dollar** — a 4.3x cost increase with zero warning.

Where Did the Tokens Go?

In February, nearly every output token wrote code. By April, **thinking rose from 1% → 18%**, and explanation ate the rest. You're being billed for the model's internal monologue — the AI equivalent of a consultant billing you for thinking about billing you.

What 10,000 Tokens Bought You

Metric	Feb	Apr	Decline
PRs shipped	0.30	0.16	47% gone
Code lines drafted	630	91	86% gone
Files touched	8.2	3.6	56% gone
Knowledge captures	0.95	0.42	56% gone

Forecasts that hold tokens-per-output constant will **drastically underestimate your spend.**

<https://thetangos.com/research/the-currency-of-coding-4-6-2024-20240501-14000-tokens-bought-you/>

<https://thetangos.com/research/the-currency-of-coding-4-6-2024-20240501-14000-tokens-bought-you/>

23

CHAPTER 2: TOKENFLATION

The Token Industrial Complex

Vendor revenue scales with **tokens burned**, not value created — a structural incentive to make models chatter longer, reason further, and generate more. During the Bigspin study, token usage **spiked across every user cohort** for a full week mid-study. No documented feature change. No session-length shift. No explanation. Even the researchers who built the CPI couldn't explain their own bill.

The Black Box Problem

If the meter can move and careful outside observers can't tell you why, ask yourself how much visibility you have inside someone else's black box.

Diminishing Returns Are Real

Doubling your token spend **does not double your accuracy**. Test-time scaling research confirms diminishing returns hit fast — more tokens ≠ better output.

The Fix: Token Leverage

Stop optimizing for token burn. Start optimizing for **token leverage** — smaller, tuned, task-specific models where they fit. Route the rest elsewhere.

<https://thetangos.com/research/the-currency-of-coding-4-6-2024-20240501-14000-tokens-bought-you/>

<https://thetangos.com/research/the-currency-of-coding-4-6-2024-20240501-14000-tokens-bought-you/>

24

CHAPTER 3: SOVEREIGNTY

The Sovereignty Illusion

We encrypt data **at rest** and **in transit** — but during inference, it sits as **plaintext in GPU memory** . We built a bank vault with a screen door on the back.

The Geographic Trap

"Data sovereignty" today = a pin on a map. Your server sits in Vizag, but the **US CLOUD Act still reaches it**. Contractual promises, geographic boundaries, compliance checkboxes — comfort blankets with **legal trapdoors sewn into the lining**.

You Can't Legislate a Memory Dump

No NDA, no audit clause, no compliance framework prevents a hypervisor-level memory scrape. The gap is technical — and legal instruments cannot patch it.

The Three-Way Trust Dilemma

1 Model Owners	2 Infra Providers
Fear weight theft and IP exfiltration	Fear liability for customer data exposure
3 You	
Fear your data in someone else's memory	

Today's fix: NDAs and audit clauses — legal instruments patching a **technical gap**.

25

CHAPTER 4: THE FIX

Confidential Computing: From "Trust Me" to "Verify Me"

Trusted Execution Environments (TEEs) are hardware-backed isolation zones that encrypt data even while it's being computed on. Root access, hypervisor, host OS — all see **ciphertext** only. Only the attested workload touches plaintext. This solves all three trust fears in one: *trustee* weights protected, code provable, data sealed.

The diagram shows three concentric circles representing layers of security. The outermost circle is orange and labeled 'Attested Workload' with the text 'Only verified code touches plaintext'. The middle circle is a darker orange and labeled 'Isolation Layer' with the text 'Ciphertext visible to host, root, hypervisor only'. The innermost circle is dark brown and labeled 'Hardware TEE' with the text 'Silicon-rooted encryption and attestation'.

The open verification stack is production-ready today: CNCF Confidential Containers · Kata Containers · IETF RATS · Keylime — cryptographic attestation checkable by anyone, not just the vendor's security team.

26

CHAPTER 4: THE FIX

Prove It, Don't Promise It

UPI didn't win because someone *promised* the banks it was secure* — it won because **the protocol was open and the rules were verifiable**. Private, verifiable AI inference is that same opportunity, one layer up the stack.

The Old Way

Tokens burned into oblivion. Blind faith in vendor promises. Sovereignty as a slogan on a slide. Compliance as a checkbox with legal trapdoors.

The New Way

Open silicon-rooted trust. Cryptographic proof instead of blind faith. One attestation receipt satisfies **DPDP, GDPR, EU AI Act and any new regulation simultaneously** — it's math, not a vendor's word.

🛑 **Stop asking your AI vendors to promise you sovereignty. Start asking them to prove it.** At Digital Public Infrastructure scale, this is the difference between sovereignty as theater and sovereignty you can hand a regulator and say: *verify it yourself*.

27

SESSION

Scaling Inference at NxtGen

Abhishek Kumar Singh

NxtGen

vLLM Inference Meetup / Bengaluru 2026

10

28

vLLM

NxtGen¹

NxtGen at Red Hat Summit Atlanta 2026

Summit sessions, labs and engineering discussions play a crucial role in establishing our engineering practices for the year for AI services.

KEY TAKEAWAYS

01

One replica efficiency

vLLM scheduling, batching and KV memory

02

Fleet intelligence

placement informed by cache, load and SLO state

03

Sovereign operations

mixed accelerators managed as one measured service



02

29

The diagram compares two customer architectures for LLM inference:

- TYPICAL GOVERNMENT CUSTOMER:** This architecture is based on a "SaaS-like Cloud Platform" for a "Government AI private cloud". It features a "Single-tenant VM application server cluster" with four "VMs" (labeled 'Application' and 'Monitoring VM'). These VMs connect to a "GPU INFERENCE SUBSYSTEM" (highlighted in red) which consists of a grid of GPU servers. Below this, the "Database and storage server tier" includes "PostgreSQL VM database servers" (labeled 'Primary', 'Secondary', 'Replica') and a "VM storage server pool" (labeled 'Storage', 'Elasticity', 'Reports', 'Backup').
- TYPICAL ENTERPRISE CUSTOMER:** This architecture is based on a "SaaS-like Cloud Platform" for an "Enterprise AI private cloud". It features a "Single-tenant VM application server cluster" with four "VMs" (labeled 'Application', 'Monitoring VM', 'GPU VM', and 'GPU VM'). These VMs connect to a "GPU VM INFERENCE SUBSYSTEM" (highlighted in red) which consists of a grid of GPU VMs. Below this, the "Database server tier" includes "PostgreSQL VM database servers" (labeled 'Primary', 'Secondary', 'Replica') and a "VM storage server pool" (labeled 'Storage', 'Elasticity', 'Reports', 'Backup').

Both architectures show a flow from the application servers through the inference subsystem to the database and storage tier. The government customer architecture is more monolithic, while the enterprise customer architecture is more modular and scalable.

30



vLLM makes one replica efficient

The engine removes waste inside a model server

01	PagedAttention	Allocates KV memory in blocks so requests do not need contiguous slabs.
02	Continuous batching	Admits and completes sequences dynamically as capacity changes.
03	Chunked prefill	Interleaves long prompts with decode work to control interference.
04	Automatic prefix caching	Reuses shared prompt blocks when the matching cache is local.

Optional levers: quantization for memory pressure, speculative decoding for decode latency, LoRA for model variants

31

vLLM

NxtGen¹

Replica efficiency does not solve fleet placement

Round-robin preserves endpoint fairness, not KV locality or latency fairness

ROUND ROBIN

Shared system prompt

vLLM pod A
prefix cached

Same prefix, new question

vLLM pod B
prefix absent

Same prefix, next turn

vLLM pod C
prefix absent

What repeats on the wire

Prefill work

computed 3 times

Useful cache

stranded by pod

Tail latency

set by cold turns

At NxtGen in 2025, we used conventional load balancing across GPU VMs serving identical models. It answered requests but offered no continuity for multi-turn or long-horizon agents.

32

vLLM

NxtGen⁷

llm-d turns the fleet into an inference control plane

Filter, score and pick with the state that determines inference cost

```
graph LR; Client[Client<br/>OpenAI API] --> Gateway[Gateway + InferencePool<br/>networking<br/>model pool contract]; Gateway --> Router[llm-d Router<br/>Endpoint Picker]; Router --> RepA[vLLM replica A<br/>best prefix match]; Router --> RepB[vLLM replica B<br/>lower load]; Router --> RepC[vLLM replica C<br/>reserve capacity];
```

Gateway API Inference Extension defines the pool. llm-d supplies the inference-aware decision.

Current ecosystem: llm-d v0.9, Gateway API Inference Extension v1.5

33

vLLM

NxtGen⁷

Model choice and replica choice are different decisions

A stable API can hide model, replica and hardware decisions without mixing their responsibilities

```
graph LR; Request[Request<br/>intent + context] --> Semantic[vLLM Semantic Router<br/>Chooses the capability path<br/>Signals can include intent,<br/>difficulty, context length,<br/>modality, identity and risk.]; Semantic --> Router[llm-d Router<br/>Chooses the execution path<br/>It evaluates prefix locality,<br/>token load, saturation,<br/>topology and latency<br/>headroom.]; Router --> vLLM[vLLM<br/>executes];
```

Earlier NxtGen solutions used rigid prompts to route agentic work to reasoning LLMs behind HA endpoints. Semantic Router and llm-d now separate model choice from replica placement.

34

vLLM

NxtGen⁷

Each scaling decision has a different success test

A single throughput number cannot validate model choice, replica placement and engine configuration

01

MODEL POLICY

Did the request reach the right capability and cost tier?

We could judge this via

Router precision and recall

Task quality by route

02

REPLICA POLICY

Did placement avoid cold cache and overloaded replicas?

We could judge this via

p95 TTFT and queue time

Cache hit and saturation

03

ENGINE + HARDWARE

Did the selected configuration improve useful capacity?

We could judge this via

Task quality and TPOT

Tokens per accelerator and cost

Benchmark the complete request path on NxtGen traffic before promotion.

35

vLLM

NxtGen⁷

NxtGen heterogeneous inference benchmark

NxtGen, IBM Research, Red Hat and evaluated two placement policies across three accelerator-vendor pools.

```
graph LR; Stream[PREFIX-RICH REQUEST STREAM<br/>The 7.2K-token prefix repeats while each<br/>question changes] --> EPP[llm-d EPP<br/>Endpoint Picker<br/>prefix-cache locality<br/>replica saturation]; EPP --> VendorA[Vendor A<br/>4 GPUs]; EPP --> VendorB[Vendor B<br/>8 GPUs<br/>warm prefix]; EPP --> VendorC[Vendor C<br/>8 GPUs];
```

1K output tokens per request

Only the routing policy changed

The published benchmark anonymizes the hardware identities as Vendor A, B and C.

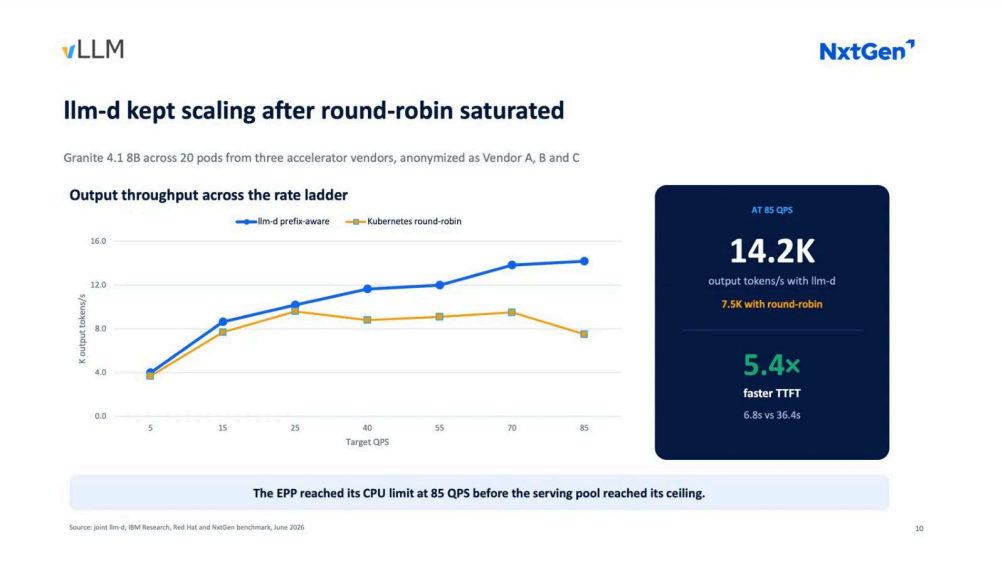
SAME PODS
both policies

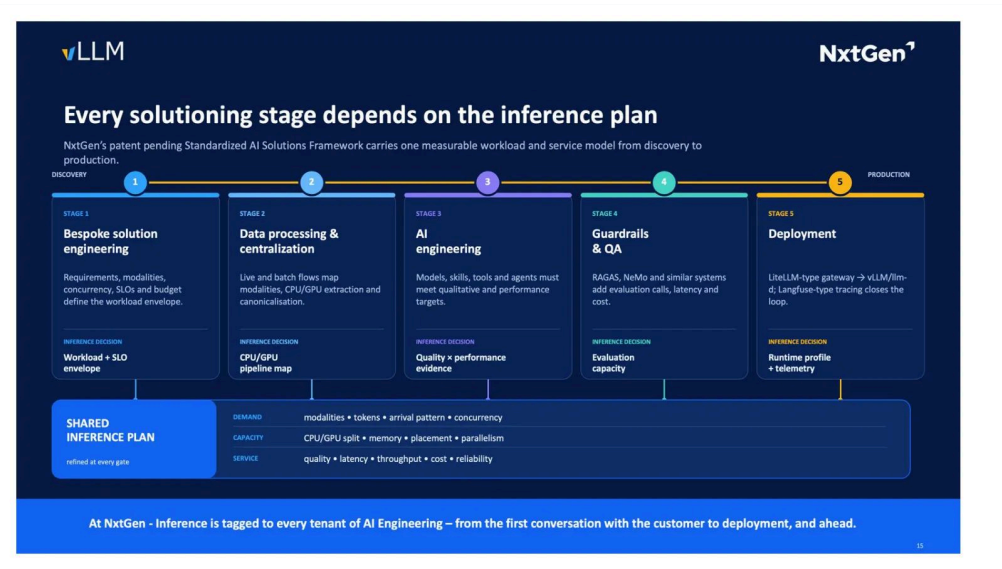
SAME IMAGES AND FLAGS
within each pool

100% ROCE
shared fabric

MODEL-SPECIFIC SCORER
precise or approximate

36

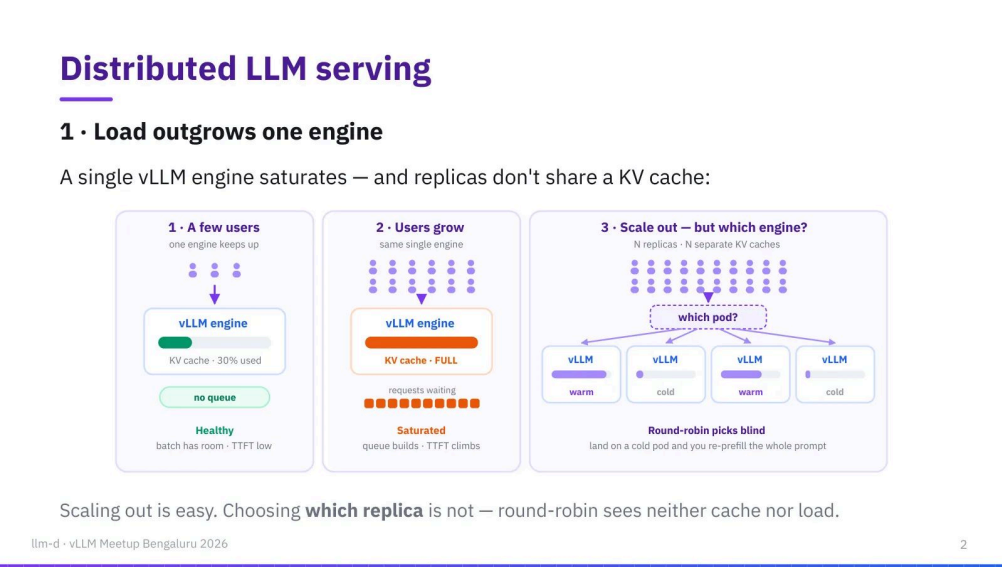




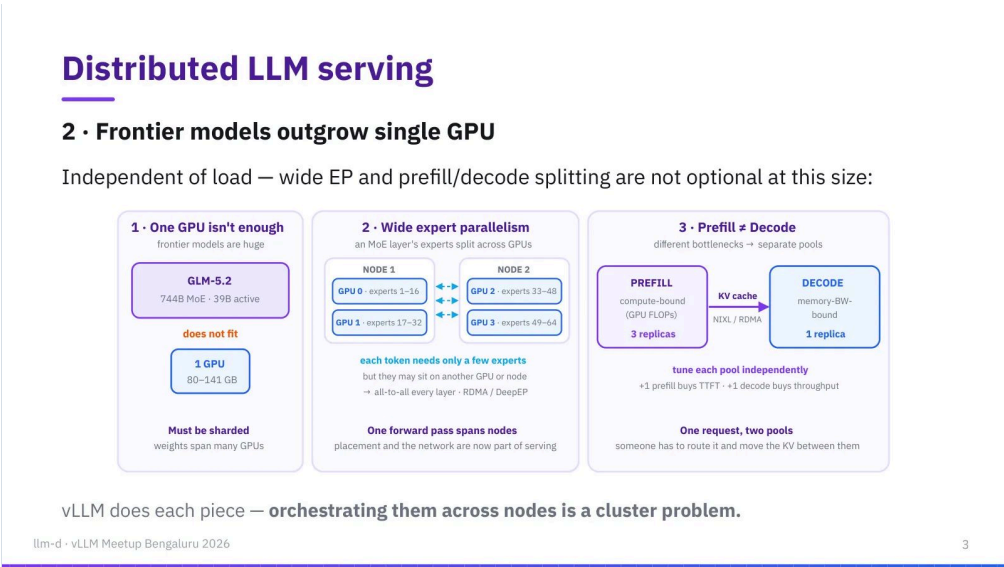
41



42



43



44

llm-d — Kubernetes, made LLM-aware

K8s-native distributed inference on **vLLM** + the Gateway-API Inference Extension — CNCF-track. **Started by IBM Research & Red Hat**, now built across the industry:

FOUNDING MEMBERS



Prefix-cache-aware routing

Prefill / decode disaggregation

SLO-aware routing

Workload autoscaling

Observability

Multi-vendor

TESLA

3× throughput
2× faster TTFT

aws

+70% tokens / sec

ORACLE

+30% throughput

Google

-40% TTFT & ITL

Real production wins, across vendors.

llm-d - vLLM Meetup Bengaluru 2026

4

45

The llm-d ecosystem

FOUNDING CONTRIBUTORS



LAUNCH PARTNERS

AMD Cisco

Hugging Face Intel

Lambda Mistral AI

CONTRIBUTORS · UNIVERSITIES

DaoCloud DigitalOcean

Moreh Rebellions

UC Berkeley · Sky

UChicago · LMCache

USERS

Capital One Tesla

Snowflake Cohere

JetBrains Prime Intellect

Reflection Stelia

Novita AI

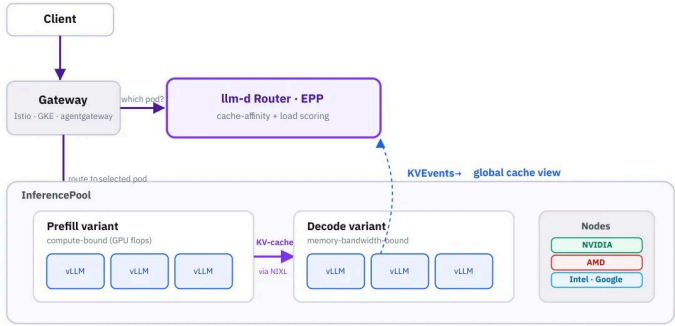
Open governance, CNCF-track.

llm-d - vLLM Meetup Bengaluru 2026

5

46

llm-d architecture at a glance



Client

Gateway
Istio - GKE - agentgateway

llm-d Router - EPP
cache-affinity + load scoring

InferencePool

Prefill variant
compute-bound (GPU flops)

Decode variant
memory-bandwidth-bound

Nodes
NVIDIA
AMD
Intel - Google

KVEvents - global cache view

KV-cache via RDMA

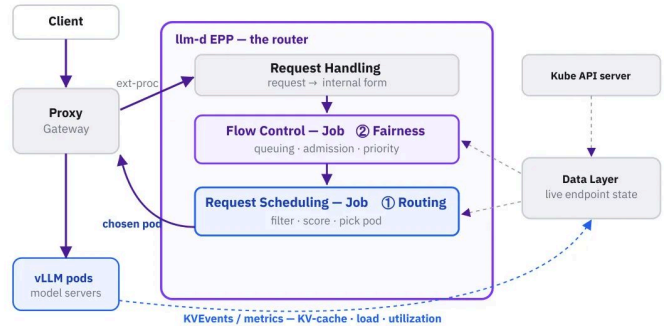
llm-d - vLLM Meetup Bengaluru 2026

6

47

The EPP does two jobs

① **Routing** — prefix- & load-aware placement · ② **Fairness** — queuing, admission & priority



Client

Proxy Gateway

llm-d EPP - the router

Request Handling
request -> internal form

Flow Control - Job ② Fairness
queuing - admission - priority

Request Scheduling - Job ① Routing
filter - score - pick pod

vLLM pods
model servers

Kube API server

Data Layer
live endpoint state

KVEvents / metrics - KV-cache - load - utilization

llm-d - vLLM Meetup Bengaluru 2026

7

48

JOB ① · ROUTING

How a pod actually gets picked

1 · PRODUCERS

Build the live view

KVEvents & metrics → who holds which prefix, how loaded each endpoint is

2 · FILTERS

Shrink the candidate set

hard constraints — role (prefill/decode), and shed endpoints that are too hot

3 · SCORERS

Rank what's left

soft preferences — weighted sum over cache hit, token load, active requests

Filters are veto. Scorers are preference. Highest total score wins the request.

All of it is **plugins** — you compose a policy.

llm-d · vLLM Meetup Bengaluru 20268

49

JOB ① · ROUTING

The same thing, as configuration

```
schedulingProfiles:  
- name: prefill  
  plugins:  
  - pluginRef: prefill-filter # FILTER · keep only prefill endpoints  
  - pluginRef: prefix-cache-affinity-filter # FILTER · shed the hot ones  
  - pluginRef: token-load-scorer # SCORER · how many tokens in flight  
  - pluginRef: cpu-prefix-cache-scorer # SCORER · can it restore from DRAM?  
- name: decode  
  plugins:  
  - pluginRef: decode-filter # FILTER · keep only decode endpoints  
  - pluginRef: active-request-scorer # SCORER · who has a free slot
```

Prefill and decode get different policies
prefill is cache- and prefill-throughput-bound, so affinity earns its keep · decode is slot-bound, so what matters is which endpoint has room right now

llm-d · vLLM Meetup Bengaluru 20269

50

JOB ② · FAIRNESS

Flow control: admission, not just placement

When **every** endpoint is saturated, you decide **who** waits:

REQUESTS
tagged with a priority + a fairness ID

PRIORITY BANDS — one queue per program

premium
standard
best-effort

first

FAIRNESS POLICY
which queue next, within a band?
Strict priority between bands
Fair share within a band
round-robin · LAS · turn-priority

SATURATION GATE
admit while there is room
hold when full
Backpressure when saturated
tokens & requests in flight

vLLM pool
endpoint
endpoint
endpoint
KV 96% · full

Routing decides where a request goes · flow control decides whether, and when

llm-d · vLLM Meetup Bengaluru 202610

51

FOR THE SREs

Deploying llm-d is just Kubernetes objects

Gateway InferencePool EPP · 1 Deployment vLLM pods · unchanged

Clients / apps
one endpoint

1 Helm chart installs all of this
Gateway + InferencePool + EPP + vLLM

Kubernetes cluster

THE ONLY NEW PIECE

llm-d router
InferencePool + Endpoint Picker (EPP)
picks a pod by cache-affinity + load

Gateway
Gateway API

Node 1
vLLM pod
unmodified · 1 GPU
GPU

Node 2
vLLM pod
unmodified · 1 GPU
GPU

Node 3
vLLM pod
unmodified · 1 GPU
GPU

cache state

Same pods, same vLLM, same Helm — **no app changes**. Only the routing layer is new.

llm-d · vLLM Meetup Bengaluru 202611

52

Agentic workloads

GLM-5.2 on 64x H200 · flow control · **#sig-agentic-inference**

Agentic serving: prefill *is* the workload

GLM-5.2 · 744B MoE · 64x H200 · replaying real Claude Code traces (219 sessions, 78K reqs)

- input/turn p50/p90/p99 = **195K / 521K / 849K** tokens
- output p50 = **317** → **98.6% of tokens are input**
- **99.6%** median prefix reuse turn-to-turn
- **107** turns/session · sub-agents burst **7–51** at once

WHY THIS BREAKS NORMAL SERVING

Context grows past **750K tokens**, then **pauses** — 2 s p50, **26 s p90**. Live session state reaches **1.4–5.6× GPU cache**, so prefixes are evicted and every returning user pays a **full recompute**.

The unit of work is a **session**, not a request — and it is **prefill-bound**.

AT PRODUCTION SCALE

The same stack on 544x H100

GLM-5.2 · **544 H100s** — not H200s · real agentic benchmarks · **zero preemptions, zero serving errors**

6.6M 3,000 5-10x

peak output tok/min

concurrent agents

cheaper than APIs

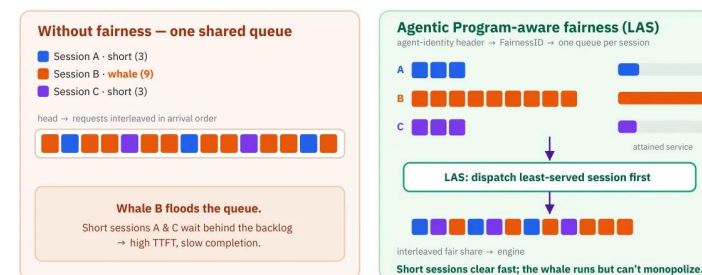


Frontier-scale open models **don't** need next-generation accelerators

IBM Research · research.ibm.com/blog/running-open-models-on-h100-gpus-with-llmd

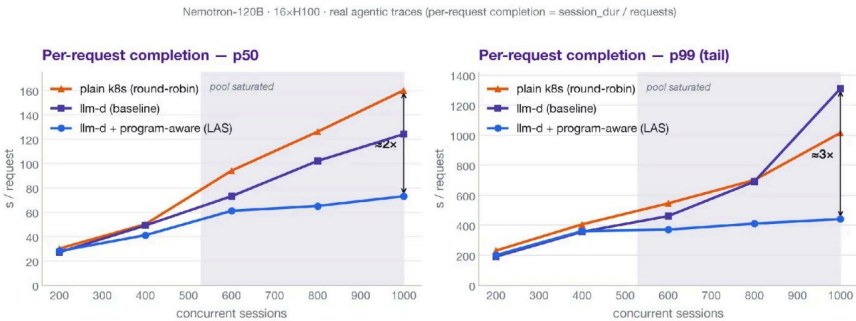
Throughput isn't the whole story

Raising pool throughput doesn't decide **who** gets served:



Standard header → **FairnessID** → one queue per session; **LAS** dispatches the least-served first.

It works: fairer, at no throughput cost



llm-d · vLLM Meetup Bengaluru 2026

21

57

FAIRNESS STRATEGIES

Turn-priority: schedule for prefix residency

Same per-program queues, one strategy switch — and the **opposite** objective:

score = turn number + (small weight × head wait) — dispatch the highest

LAS · EQUITY

Least-served session first

unit = weighted tokens — rescues short sessions stuck behind a whale

TURN-PRIORITY · EFFICIENCY

Deepest session first

unit = session depth — a deep session's prefix is still resident, so serving it re-prefills nothing

On the agentic replay: ~2× throughput — repeated prefill comes back as capacity

Under contention it **sheds new sessions** rather than lose a deep prefix.

llm-d · vLLM Meetup Bengaluru 2026

22

58

WHAT'S NEXT

The agentic roadmap

- 1 **Session-aware routing** — keep a session on the pod that already holds its history
- 2 **KV-cache retention** — hold a session's cache across turns; evict by session state
- 3 **Session-graph scheduling** — schedule whole agent workflows, not single requests
- 4 **Agent hints** — let the agent signal intent to steer routing, caching & priority

The unit keeps growing: **requests → agents → sub-agents → multi-agents.**

llm-d · vLLM Meetup Bengaluru 2026

23

59

Thank you — questions?

Get involved: llm-d.ai/community · github.com/llm-d

#SIG-Agentic-Inference is driving the agentic work — come build it with us.

llm-d.ai/blog

Pravein Govindan Kannan · pravein.govindan.kannan@ibm.com · github.com/praveingk

60

SESSION

Distributed Inference on ROCm

WideEP on vLLM & llm-d

Chaitanya Sri Krishna Lolla & Sirra Ajith

AMD

vLLM Inference Meetup / Bengaluru 202607

61

AMD

Meet the Speakers



Chaitanya Sri Krishna Lolla

Manager, Software Development — ML Frameworks Group, AMD

Leads the AI Model Performance and Distributed Inference India team, driving deep performance analysis and optimization across AI frameworks.



Sirra Ajith Prakash

Member of Technical Staff — AMD

Works on large language models, optimizing inference stacks such as vLLM on AMD Instinct MI GPUs, benchmarking agentic workloads, and profiling execution to identify bottlenecks.



Aditya Sivagnanam

Developer Relations, AIG Team — AMD

Over a decade of expertise working with customers as a Field Applications Engineer at AMD.

AMD

together we advance.

62

AMD AI Developer Program

Build Faster Learn Deeper Scale Smarter

AMD AI Developer Program Benefits

Cloud Credits

Start building with free cloud credits from **AMD Developer Cloud credits** or **Fireworks AI credits** when you join.

Community Access

Join our **Discord** and **forums** to get help, share ideas, and dive into technical conversations.

Learning Resources

✓ Explore **AI Academy** to access expert-led and self-paced AI courses.

✓ Enjoy one-month complimentary DeepLearning.AI pro membership.*

Showcase Opportunities

Share your projects for a chance to be featured on AMD Developer channels and recognized at AMD events

AI Developer Program provides the compute, expertise, and community developers need to accelerate AI innovation

*Terms and conditions apply see <https://www.amd.com/en/developer/ai-dev-program.html#faq>.

AMD

together we advance.

63

AMD AI Developer Program

Who Should Join?

The AMD AI Developer program is built for developers who:

- Build or train AI models
- Optimize compute-intensive workloads
- Work with open-source frameworks
- Explore AI acceleration on AMD hardware

How to Join?

- Create your free AMD account and [complete the AI Developer Program form](#)
- Verify your email using the activation code and access the [AMD AI Developer program portal](#)
- Join the community channels to start connecting with fellow developers
- [Submit the form](#) to request your cloud credits to choose between AMD Developer Cloud credits or Fireworks AI Credits
- Stay tuned for member emails about events, training, and new resources



AMD

together we advance.

64

AMD AI Academy

Self-learning courses

AI Foundations

Build a strong foundation in artificial intelligence, machine learning, and generative AI concepts while learning how to run and experiment with AI models on AMD platforms.

8 Courses 15 Hours JL 108

[View Pathway](#)

Build AI

Learn how to build intelligent AI-powered applications, agents, and multimodal workflows using modern frameworks, open-source models, and scalable AI architectures.

5 Courses 8 Hours JL 377

[View Pathway](#)

Deploy & Optimize AI

Deploy, scale, and optimize AI systems for production using AMD GPUs, modern inference frameworks, Kubernetes, and GPU performance optimization techniques.

6 Courses 7 Hours JL 205

[View Pathway](#)

Academic Developers

Learn how to empower teaching and research with AI through hands-on tutorials and modern techniques that enhance coursework, support students, and advance research.

7 Courses 14 Hours JL 219

[View Pathway](#)

<https://www.amd.com/en/developer/resources/training/amd-ai-academy.html>

AMD together we advance.

65

AMD AI Academy example Course Curriculum

Introduction To AI Agents

2 Hours 12 Lessons 4 Projects 200 Points

Course Curriculum

- Module 1: Introduction To AI Agents 1 Lessons
 - Video: Introduction To AI Agents 1 Lesson
- Module 2: Components And Architecture Of AI Agents 1 Lessons
 - Video: Components And Architecture Of AI Agents 1 Lesson
- Module 3: The Orchestration Layer 1 Lessons
 - Video: The Orchestration Layer 1 Lesson
- Module 11: Hands On Lab 4 1 Lessons
 - Lab: Router Agent 1 Lesson
- Module 12: Quiz 0 Lessons

Unenroll

Duration

2 Hours

Lessons

12 lessons

Projects

4 hands-on

Level

Beginner

Certificate

Yes

AMD together we advance.

66

AMD ROCm Certification Program

ROCm Certified Programs

The ROCm Certified Associate Digital Badge and Certificate will be awarded by email via Credly upon successful completion of the certification.

Available Now

ROCm™ Certified Associate

Core AMD ROCm™ Software Application Development (Associate)

Learn how to design software application code built around hands-on workshops focused on the design, optimization, and deployment of AI on AMD Instinct™ GPUs.

Coming soon

ROCm™ Certified Professional

Advanced AMD ROCm™ Software Development and Production Readiness (Professional)

Gain certification validating your ability to design, optimize, and deploy production-scale distributed AI training and high-performance inference workloads across multi-GPU and multi-node environments.

Coming soon

ROCm™ Certified Expert

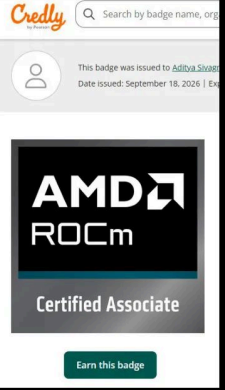
Architecture-aware AMD ROCm™ Software Performance Optimization and Scale (Expert)

Learn how to design, deploy, and operate production AMD GPU infrastructure, from scaling AI workloads and GPU pipelines to managing and automating GPU clusters in real-world environments.

Certification Overview

Core AMD ROCm™ Software Application Development - Certified Associate





AMD together we advance.

67

Start your AI journey with AMD Developer Playbooks™

Start your AI journey with AMD Developer Playbooks™

Step-by-step playbooks and powerful AI models ready to run. Choose your device to get started.

AMD Ryzen™ AI Hubs Ryzen™ AI APUs Radeon™ GPUs AI



Generating Images with ComfyUI and Z Image Turbo

Create stunning AI-generated images using ComfyUI with Z Image Turbo on RTX Hubs™

15 min Start Learning

Automating Workflows with vllm and Local LLM

Automate AI-generated text generation using vllm and open-source LLMs on RTX Hubs™

15 min Start Learning

Local LLM Coding with VRCode and OpenAI Code

Run AI-generated code locally using VRCode and OpenAI Code on RTX Hubs™

15 min Start Learning

Running and serving LLMs with LLaMA Studio

Run and serve LLaMA models locally using LLaMA Studio on RTX Hubs™

15 min Start Learning

Running LLMs with PyTorch and ROCm

Run LLaMA models locally using PyTorch and ROCm on RTX Hubs™

15 min Start Learning

Building Your First Agent with GPT

Build your first AI agent using GPT and OpenAI on RTX Hubs™

15 min Start Learning

Clustering Ten RTX Hubs with Remote-ops IPC

Run AI workloads across multiple RTX Hubs using Remote-ops IPC on RTX Hubs™

15 min Start Learning

<http://playbooks-dev.amd.com/#playbooks>

AMD together we advance.

68

AMD University Program (AUP) AI & HPC Cluster for Researchers

Slurm cluster comprised of AMD EPYC CPUs and AMD Instinct GPUs (MI210, MI250, MI300X, MI325X, MI350X) connected with a GPU-aware interconnect.

- ROCm for running AI & HPC workloads on Instinct
 - Heterogeneous-Computing Interface for Portability (HIP)
 - Familiar CUDA-like syntax (e.g., hipMalloc, hipMemcpy) to develop portable source code for AMD and NVIDIA accelerators
 - hipify tools for automatic translation from CUDA to HIP
 - Profiling/debugging tools, GPU-accelerated libraries, etc.
- Parallel file system for IO and persistent storage
- ROCm-enabled PyTorch
 - Runs out of the box on ROCm with Day 0 support

Node-hour allocations available to academic researchers and educators

- Pls must be academic faculty or postdocs
- Application-level work (e.g., no sudo)



<https://www.amd.com/en/corporate/universityprogram/ai-hpc-cluster.html>



AMD together we advance_

69

Developers: AMD Developer Cloud

Pay-as-you-go platform to provide easy access to 1x- or 8x-MI300X GPU VMs for developers

- Compute credits available to get started
- Docker containers (pre-installed or custom) available to simplify setup
- Snapshots to save progress and restart where you left off

Two AMD Instinct MI300X GPU Instance Configuration

Small Configuration: 1 x GPU	Large Configuration: 8 x GPU
• GPU Memory: 192 GB • vCPUs: 20 vCPUs • System Memory: 240 GB • Boot Disk: 720 GB NVMe • Scratch Disk: 5TB- NVMe	• GPU Memory: 1536 GB • vCPUs: 160 vCPUs • System Memory: 1920 GB • Boot Disk: 1920 GB NVMe • Scratch Disk: 40 TB- NVMe

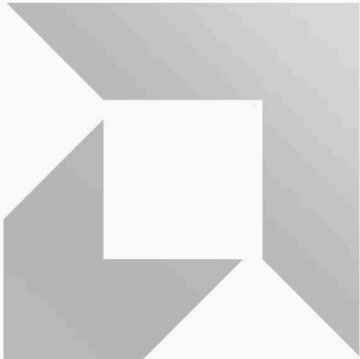
Create an Account





AMD together we advance_

70



Distributed Inference on ROCm with WideEP on vLLM & llm-d

Chaitanya Sri Krishna Lolla
Manager Software Development

AMD together we advance_

71

Public

Acknowledgements

Mathews Aswin, Sun Peng, Li Chuan, Ainapurapu Satya Ramji, Ravi Gupta, Shiksha Patel, Roche Kenny, Cave Vincent, Douglas Lehr, Ai Fangzhou, Li Li, Akbarzadeh Amir, Tej Kiran, Avinash Paul, Nitin Singh, Mir Ali, Barakat Basem, Prateek Kumar

AMD together we advance_

72

Public

What is distributed inference?

- One model, many GPUs and many nodes — it no longer fits, or keeps up, on one
- Prefill and decode want opposite things — split the pools, move the KV cache
- Experts spread across the cluster — every layer pays an all-to-all

Two pools, one KV hop, an all-to-all in every MoE layer.

AMD

together we advance.

73

Public

The shape of MoE communication

- Dispatch: token to its expert's GPU. combine: partial result back to the home GPU
- Internode normal kernels for prefill throughput; low-latency IBGDA kernels for decode

Two all-to-alls per MoE layer, per token, on the critical path.

DeepEP and MoRI-EP fuse routing, XGMI and RDMA into exactly these two kernels

AMD

together we advance.

74

Public

Wide EP: DP attention + EP MoE

- **Attention runs data-parallel** — each rank owns its requests and their KV. No cross-rank attention traffic, no KV duplication.
- **MoE runs expert-parallel** — experts spread over *all* ranks
- One communication pattern instead of two competing ones
- vLLM “*--data-parallel-size*” + “*--enable-expert-parallel*”
- All-to-all backend on ROCm: “*--moe-a2a-backend mori_high_throughput | mori_low_latency*”

AMD together we advance.

75

Public

Why “wide”, and what it costs

- **Wide** = the expert-parallel group spans **many nodes**
- This is the direct answer to “why multi-node?”
- More GPUs → fewer experts each → more HBM free for KV cache and batch size
- **The cost**: the all-to-all now crosses the network, and its participant count grows
- MoRI-EP dispatch: 307 GB/s XGMI at EP8; at EP16, 171 GB/s XGMI + 52 GB/s RDMA [1]

[1] MoRI-EP dispatch benchmarks — github.com/ROCm/mori/benchmarks
Configuration: MoRI-EP dispatch/combine microbenchmark, 8x AMD Instinct™ MI300X GPU per node with ConnectX-7 NICs; EP8 = 1 node, EP16 = 2 nodes; 4096 tokens, 7168 hidden, top-8 experts, FP8 dispatch / BF16 combine, DeepSeek-V3 configuration. Results may vary.

AMD

together we advance.

76

Public

The ROCm distributed inference stack

AMD together we advance

77

77

Public

ROCm Distributed Inference Framework Software Stack

Three communication bands, not one — KV transport, expert parallel and shared memory get conflated constantly; the rest of this section takes them one at a time, bottom up

orchestration profile, routing, allocating	llm-d	ATOMesh	
frameworks batching, scheduling, paged KV	vLLM	SGLang	ATOM
kernel library fused MoE, attention, GEMM	AITER		FlyDSL
KV transport prefill → decode, point-to-point	MoRI-IO	Mooncake	NIXL
expert parallel dispatch / combine all-to-all	MoRI-EP		DeepEP
shared memory GPU-initiated put / get	MoRI-SHMEM	rocSHMEM	RCCL
NIC must support GPU-directed RDMA	AMD AI NIC	Thor2	CX7
hardware HBM capacity sets EP width	MI300X	MI325X	MI355X

same stack whether you run vLLM, SGLang or ATOM — only the top two bands change

AMD together we advance

78

78

Public

AITER

- AMD's ROCm kernel library – fused MoE, attention, GEMM, RMSNorm and quantization ops
- Four kernel tiers – HIP C++, Triton, FlyDSL and hand-written assembly – behind one Python/C++ op API
- Also loads under vLLM and SGLang as a ROCm kernel backend, so engine code stays unchanged
- Fused kernels remove the HBM round trips between routing, permute, GEMM, activation and scaling
- Per-arch tuned configs for MI300X/MI355X, selected at runtime by shape, dtype and expert count
- Block-scaled FP8 and INT4 paths with scaling folded into the GEMM epilogue, plus paged-attention and MLA kernels

WITHOUT AITER — op by op

WITH AITER — one fused kernel

AMD together we advance

79

79

Public

MoRI — a layered stack, not a monolith

ONE STACK, FOUR LAYERS — EACH ONE USABLE ON ITS OWN

vLLM · SGLang · ATOM
the frameworks that call in — MoE layers and PD KV transfer

MoRI-EP
dispatch / combine all-to-all
one kernel, XGMI inside the node and RDMA across nodes

MoRI-IO
point-to-point KV transfer
GPU-initiated RDMA writes, layer by layer

MoRI-CCCL
lightweight collectives
runs on SDMA, stays off the compute units

MoRI-SHMEM
the foundation — symmetric memory and GPU-side put/get that all three layers are built on

XGMI inside the node · RDMA across nodes · SDMA for the quiet path

take the whole stack, or take one layer — MoRI-IO alone is a valid KV connector

AMD together we advance

80

80

Public

MoRI-EP + AITER fused MoE — one layer end to end

- One MoE layer is three things: dispatch all-to-all, expert GEMM, combine all-to-all
- MoRI-EP owns both all-to-alls — one kernel, XGMI in-node and RDMA across
- AITER fuses the expert side — no HBM round trips between routing, GEMM and scaling
- Split send/rcv overlaps dispatch for micro-batch N+1 with the GEMM for N; kernel choice is the knob — InterNodeV1 for prefill, V1LL/AsyncLL for decode

serial

overlapped

time saved per layer

comm and compute run on separate queues — MoRI-EP dispatch/combine overlaps the GEMM

AMD

together we advance.

81

Public

Recent upstream work

AMD

together we advance.

82

Public

Lessons from scaling GLM-5.1-FP8 to 64 GPUs

- Full prefill/decode disaggregated Wide-EP serving for GLM-5.1-FP8 on MI300X, over AMD MoRI
- Model shape: 78 layers, 256 experts top-8, MLA plus DeepSeek Sparse Attention
- Scaled to 64 MI300X GPUs — EP8 through EP32, all-to-all on both the prefill and decode side [1]
- Two upstream enablement items landed first: per-request attention metadata, and full indexer KV transfer
- Hardware: MI300X (gfx942), 8 GPUs per node, RoCEv2 fabric

WHAT IT TOOK — FOUR PIECES

Model Enablement
MLA and DSA attention running under Wide-EP disaggregation, not just single-node.

Attention Metadata
vLLM #47766 keys metadata per request, keeping the persistent sparse-MLA kernel on the fast path

Indexer KV transfer
the connector transfers all 156 caches, 78 main plus 78 indexer

Transport
MoRI-EP for the two all-to-alls, MoRI-IO for the RDMA KV hop between pools

the enablement is upstream, not a private branch — this runs on the stock vLLM ROCm image plus MoRI

[1] ROCm blog, "DI Series: Scaling GLM-5.1-FP8 to 64 MI300X GPUs" — rocm.blogs.amd.com/software-tools-optimization/di-glm-wideep

AMD

together we advance.

83

Public

GLM-5.1-FP8 — the complete 72/72 benchmark matrix

Four topologies, three input shapes, six concurrencies. All 72 cells measured; all four topologies met the NIAH pass threshold from 2k to 35k. [1]

THE SWEEP

prompt

harness NIAH + perf

1P / 1D

2P / 2D

2P / 4D

4P / 4D

EP8 → EP32

72/72

4 topologies × 3 shapes × 6 concurrencies

RESULTS — NIAH PASS THRESHOLD MET

Topology	NIAH low scoring cell	Result
1P / 1D	9 / 10	pass
2P / 2D	8 / 10	pass
2P / 4D	8 / 10	pass — 16,525 toks/s
4P / 4D	7 / 10	pass
2k → 35k	all widths	threshold met
72/72 cells	measured	complete

The 7-9/10 dps are single-measure variance at a fixed seed. Retrieval met the NIAH pass threshold across the full 2k-35k range at each EP width tested, on both the prefill and decode all-to-all.

[1] ROCm blog, "DI Series: Scaling GLM-5.1-FP8 to 64 MI300X GPUs" — rocm.blogs.amd.com/software-tools-optimization/di-glm-wideep
Configuration: 5x AMD Instinct™ MI300X GPU (gfx942) per node, RoCEv2 fabric, eight GPU-LOCAL backend NICs, vLLM built from source (wide-EP WRITE mode + PR #47766), AITER v030a040, MoRI 42a895472b08; GLM-5.1-FP8. Benchmark results are preliminary; actual performance may vary based on configuration, software versions, drivers, and optimizations.

AMD

together we advance.

84

Public

KV Capacity sets the concurrency ceiling

GLM-5.1-FP8 KV capacity envelope — max context vs concurrency (from measured 557k tok/rank pool; capacity linear in decode instances)

Fig 1 — KV capacity envelope: max context vs. concurrency by topology [1]

Total throughput — ISL/OSL 28k/1k

Fig 2 — Decode throughput at 28k/1k as concurrency rises [1]

The drop is not a mystery — it is arithmetic. vLLM reports the measured per-rank KV pool (657,000 tokens per decode rank), and GLM-5.1's per-token KV cost is fixed: MLA latent (576 B) plus the DSA indexer (128 B), across 78 layers, is 53.6 KiB per token in FP8. So the longest context a topology can keep resident for all in-flight sequences has a closed form:

$$\text{max_context} = \frac{657,000 \times 8 \times (\text{decode instances})}{\text{concurrency}}$$

tokens, full pool
= 5.26M × (decode instances) / concurrency capped at the model's 202k limit

[1] Fig 1 and Fig 2 — ROCm blog, "DI Series: Scaling GLM-5.1-FP8 to 64 MI300X GPUs" — rocm.blogs.amd.com/software-tools-optimization/di-glm-wideep

AMD
together we advance.

85

Public

Sizing the decode pool by KV capacity

- **At 28k input and 256 concurrent requests**, a single decode instance sustains **1,405 tok/s**
- The driver arithmetic: the KV working set is 7.34M tokens; one decode pool holds 5.26M
- **Adding decode instances recovers the order of magnitude: 2P/4D peaks at 16,525 tok/s on the same shape**
- **A closed-form capacity model** matches where the crossover appears, and predicts each topology's long-context reach
- The rule: size the decode pool by KV capacity, not by FLOPs — long context is a memory problem
- Validated against measurement across all four topologies in the 72-cell matrix [1]

ONE DECODE INSTANCE VS. A DECODE POOL

1 decode instance
1,405 tok/s — working set exceeds pool capacity

4 decode instances
16,525 tok/s — the working set fits

+ KV pool capacity

11.8x throughput

28k input, 256 concurrent — the crossover is predicted before it is measured

[1] ROCm blog, "DI Series: Scaling GLM-5.1-FP8 to 64 MI300X GPUs" — rocm.blogs.amd.com/software-tools-optimization/di-glm-wideep
Configuration: 8x AMD Instinct™ MI300X GPU (g942) per node, RoCv2 fabric, eight GPU-local backend NICs, vLLM built from source (wide-EP WRITE mode + PR #47766), AITER e035a6040, MoRI 42a895472b08; GLM-5.1-FP8. Benchmark results are preliminary; actual performance may vary based on configuration, software versions, drivers, and optimizations.

AMD
together we advance.

86

Public

vLLM DI CI — what the nightly matrix covers

- An upstream DI CI pipeline for ROCm — disaggregated prefill/decode over the MoRI-IO KV connector & MoRI EP, on Buildkite plus Slurm
- RFC #48117 sets the plan and the test matrix; PR #47030 is the initial implementation, now merged [1]
- Gated on end-to-end accuracy, not just exit codes — GSM8K run per topology
- The multi-node RDMA path gets automated nightly coverage
- Every test group lands as a row on a public status dashboard, refreshed nightly

FOUR AXES OF THE MATRIX

Topologies

- 1P/1D and 2P/2D splits
- Single-node and multi-node
- vLLM Router in the loop

Parallelism

- TP8 and DP8/EP8 controls
- Wide EP DP16/EP16
- Spanning hosts

Hardware & Fabric

- MI305X node class
- AMD AI NIC fabric
- 8 NICs per node, one per GPU

Models

- DeepSeek V3, DSR1-MXFP4
- GLM-5.2-FP8, Kimi-K2.5-MXFP4
- MiniMax-M3-FP8

one pipeline, four axes — a test group is a point in this matrix, and the dashboard is the matrix rendered nightly

[1] vLLM RFC #48117 and PR #47030 — github.com/vlm-project/vlm — and the DI CI pipeline, buildkite.com/vlm/amd-distributed-inference-ci

AMD
together we advance.

87

Public

Where the matrix stands today — vLLM DI CI

- **The controls are clean** — TP8 and DP8/EP8 disaggregated runs pass consistently on ROCm
- EP8/DP8 intranode sits at 30/30 passing [1]
- DP16 internode is validated across four models and PRs in review
- **Wide TP** coverage is in the matrix
- **WideEP DP16/EP16**: under active development
- Nightly results are triaged and owned, with upstream issues tracked
- MoRI upgrade and API changes landed in vLLM #56073, with router support for remote TP size

HOW A NIGHTLY RESULT IS TRIAGED

in development
Wide EP DP16/EP16 with MTP at high concurrency

nightly triage
Triage and owned the same night

upstream change

tracked — vLLM #45749

[1] vLLM AMD distributed inference CI — buildkite.com/vlm/amd-distributed-inference-ci/builds/60

AMD
together we advance.

88

Public

llm-d Wide EP with MoRI on ROCm

- Wide EP now supported on llm-d end-to-end workflow on ROCm — LeaderWorkerSet, 2P2D
- The enablers are merged: vLLM #45043 and llm-d router #2094, with deployment guides in #1735 [1]
- MoRI sits underneath — MoRI-IO 1P1D added as a new test, MoRI-EP and MoRI-IO are covered by the nightly matrix
- Both routing paths are covered nightly — precise prefix cache and predicted latency routing both pass

WIDE EP ON llm-d — NIGHTLY LWS 2P2D ON ROCm NODES

the Wide EP recipe you would hand-launch is now a LeaderWorkerSet the nightly workflow brings up, benchmarks and tears down — on the same vLLM ROCm image

[1] vLLM PR #45043 — github.com/vllm-project/vllm — and llm-d router PR #2094, deployment guides PR #1735, nightly ROCm e2e workflows — github.com/llm-d

AMD
together we advance.

89

Public

Where the matrix stands today — llm-d on ROCm

- Wide EP 2P2D passes nightly
- KV-cache offloading is landing end to end (PR #2464), so capacity stops being bounded by HBM [1]
- Prefix cache and predicted latency routing are passing
- Each track lands as a well-lit path plus a nightly workflow, not a one-off recipe

FOUR PUBLIC WORKSTREAMS

Wide EP

- Beyond LWS 2P2D (wide-ep-lws guide)
- Workload autoscaling — PR #2469

KV capacity

- KV offloading end to end — PR #2464
- Tiered cache past HBM (tiered-prefix-cache)
- Longer context, same footprint

Smart routing

- Prefix cache aware today
- Predicted latency routing guide

Fabric

- RDMA fabric with GPU-direct support
- Covered by the nightly matrix
- MoRI-IO via the llm-d router

Each path ships with a nightly workflow you can run

[1] llm-d well-lit paths and nightly ROCm end-to-end workflows — github.com/llm-d — KV offloading PR #2464, autoscaling PR #2469

AMD
together we advance.

90

Public

Resources

Blogs & documentation

Disaggregated inference with GLM + Wide-EP
rocm.blogs.amd.com/software-tools-optimization/di-glm-wideep

Wide expert parallelism for DeepSeek
rocm.blogs.amd.com/software-tools-optimization/wide-ep-deepseek

vLLM + MoRI recipe (ROCm docs)
rocm.docs.amd.com/projects/ai-ecosystem/en/vllm-mori-recipe.html

MoRI-IO KV connector for vLLM
vllm.ai/blog/2026-04-01-mori-io-kv-connector

Prefill / decode disaggregation on ROCm
rocm.blogs.amd.com/software-tools-optimization/disaggregation

End-to-end and fine-grained tracing in llm-d
llm-d.ai/blog/end-to-end-and-fine-grained-tracing-in-llm-d

Quick DI recipes on AMD GPUs (ROCm/MAD)

SQLang disaggregated serving script
github.com/ROCm/MAD/tree/develop/scripts/sqlang_disagg

vLLM disaggregated serving script
github.com/ROCm/MAD/tree/develop/scripts/vllm_disagg

Where to start: read the disaggregation blog for the concepts, then run the MAD scripts to reproduce a working prefill/decode setup end to end.

91

Public

Developer resources

Get hands on with AMD Instinct GPUs — scan a code or use the link below it.

AMD Developer Cloud

Spin up MI300X instances in minutes — \$100 in complimentary credits for 30 days, no local hardware needed.

www.amd.com/en/developer/resources/cloud-access/amd-developer-cloud.html

AMD AI Developer Program

Free, self-paced AI Academy learning paths, tutorials and developer blogs for building on ROCm.

developer.amd.com/amd-ai-academy/

AMD
together we advance.

92

Public

93

COPYRIGHT AND DISCLAIMER

©2026 Advanced Micro Devices, Inc. All rights reserved.

AMD, the AMD Arrow logo, AMD AI NIC™ adapter, AMD Instinct™ GPU, ROCm™ software and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AMD

together we advance.

3:30–4:00 PM

Coffee & Snacks

Sessions resume at 4:00 PM



vLLM Inference Meetup / Bengaluru 2026

08

SESSION

Inside vLLM Semantic Router

Intelligent, cache-aware routing based on request semantics, model capabilities, workload characteristics, and infrastructure constraints.

Aayush Saini

Red Hat

vLLM Inference Meetup / Bengaluru 2026

<https://aadeshai.xyz/presentation-vllm-sr/>

Thank you!

vLLM Inference Meetup / Bengaluru 2026